Universidad Autónoma de Santo Domingo
Facultad de Ciencias · Maestría en Ciencias de Datos e Inteligencia Artificial
Implementación de Robots y Máquinas Inteligentes (Capstone Projects)

Bedeles
Asistencia Universitaria Inteligente

Informe de la Fase 2 · MVP funcional, evaluación con métricas por subgrupo, reflexión ética y canvas actualizado

Maestrantes — Grupo 3:
Fernielis Vicioso Rodríguez · Johan Sánchez · Julián Alejandro Núñez Toribio
Michael Rafael Ventura Bautista · Miguel Ángel Consoro Germán
Paola Melissa Ortega Burgos · Scarlo Yasiell Brito

Maestro: José Aquino
Santo Domingo de Guzmán, D.N. · Octubre de 2026

1. Resumen ejecutivo

Este informe documenta la Fase 2 del proyecto Bedeles, un asistente digital para servicios de apoyo universitarios (orientación, incidencias y avisos). La fase contempla cinco productos: (1) un repositorio reproducible con código y README; (2) una demo funcional en Streamlit que se puede usar en vivo; (3) un informe de evaluación con métricas desglosadas por subgrupo; (4) una reflexión ética sobre sesgos ligada a las mediciones del prototipo; y (5) el Business Model Canvas corregido con lo aprendido. El prototipo construido responde consultas frecuentes por recuperación (TF-IDF), registra incidencias con ticket trazable y muestra un panel de métricas.

Resultado central de la evaluación: sobre 65 preguntas de prueba que el motor nunca vio durante su construcción, el asistente recupera la FAQ correcta en el 64.2% de las consultas dentro del catálogo (respuesta automática: 79.2%, de las cuales 15.1 puntos son respuestas incorrectas), se abstiene correctamente ante el 58.3% de las preguntas fuera de alcance y responde en menos de 1 milisegundo ya cargado. La brecha más visible del conjunto de prueba es por tipo de pregunta: las casi idénticas al catálogo se recuperan al 100% y las paráfrasis coloquiales al 54.5% — una señal exploratoria (conjunto sintético pequeño) que alimenta la reflexión ética y el plan de mejora.

Estado de la entrega: el repositorio, la demo, la evaluación y este informe están preparados localmente como prototipo de referencia para revisión del equipo; la publicación en GitHub, el despliegue en la nube, la carpeta de Drive y el canvas v2 en Canvanizer son pasos pendientes que el equipo debe completar antes de la entrega final (sección 9).

2. Definición de alcance del MVP (propuesta)

El objetivo del MVP es demostrar, con un prototipo funcional y medible, que un asistente digital puede atender una parte de las consultas rutinarias de servicios universitarios y registrar las incidencias del campus con trazabilidad, reduciendo la dependencia de la atención presencial para los casos más simples. Esta delimitación de alcance se presenta como propuesta para revisión y ajuste del equipo:

Dentro del alcance	Fuera del alcance (fases futuras)
Asistente de consultas (chat web) con 25 FAQs: aulas, horarios, trámites, servicios y avisos	**Aplicación móvil nativa y kioscos físicos**
Abstención honesta: sin confianza suficiente, orienta hacia la atención humana	**Integración con SIS/LMS, pagos o mensajería (WhatsApp, SMS)**
Reporte de incidencias con categoría, edificio, prioridad y número de ticket (SQLite)	**Mapa interactivo del campus y localización en tiempo real**
Consulta del estado de una incidencia por número de ticket (estado inicial)	**Asistente generativo (LLM), multiidioma y autenticación de usuarios**
Panel de métricas: consultas, tasa de respuesta automática e incidencias	**Operación con datos reales de una institución (el catálogo usa el campus ficticio Universidad Nueva Aurora)**
Evaluación reproducible: 90 preguntas etiquetadas con métricas por subgrupo	**Datos personales de usuarios (no se solicitan por diseño)**

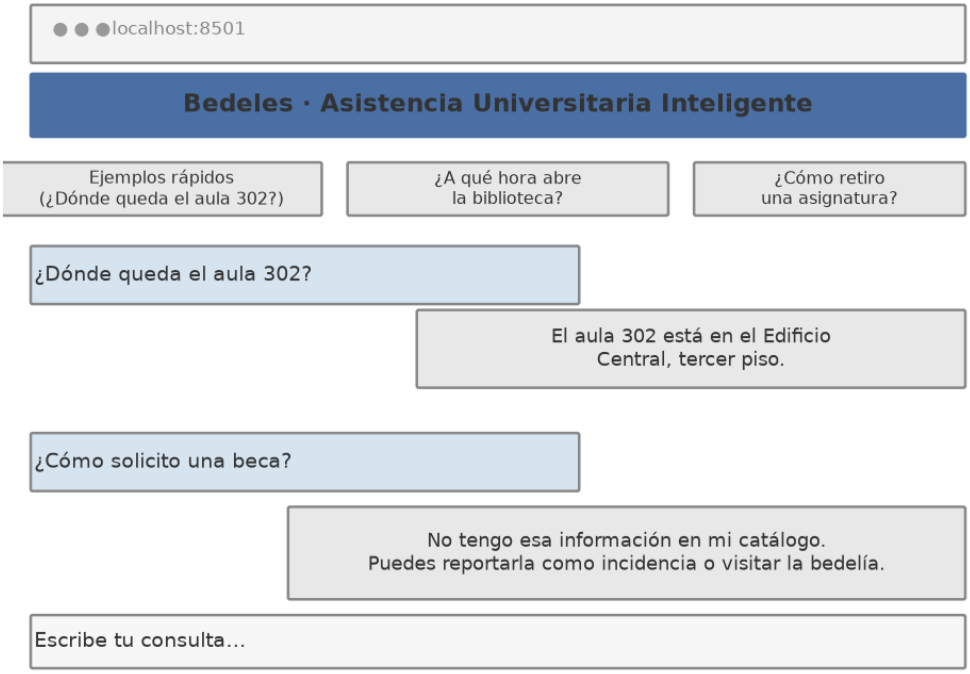
Criterios de referencia

Criterio (propuesta de referencia)	Meta	Resultado medido
Demo ejecutable en vivo	**Correr localmente en menos de 5 minutos**	**Cumplido (3 dependencias, sin claves de API)**
Acierto en preguntas dentro del catálogo	**≥ 60% en el conjunto de prueba**	**64.2%**
Abstención correcta fuera del catálogo	**≥ 50% en el conjunto de prueba**	**58.3%**
Latencia del motor	**< 1 segundo**	**0.8 ms promedio (motor ya cargado)**
Trazabilidad de incidencias	**Ticket consultable**	**Cumplido (estado inicial; transiciones contempladas en el diseño)**

3. Diseño conceptual de pantallas (mockups)

Los mockups muestran la guía visual de lo que el prototipo implementa: cuatro pantallas no funcionales que luego se construyeron en Streamlit. Representan el MVP, no una plataforma mayor; kioscos, app móvil e integraciones quedaron fuera del alcance propuesto.

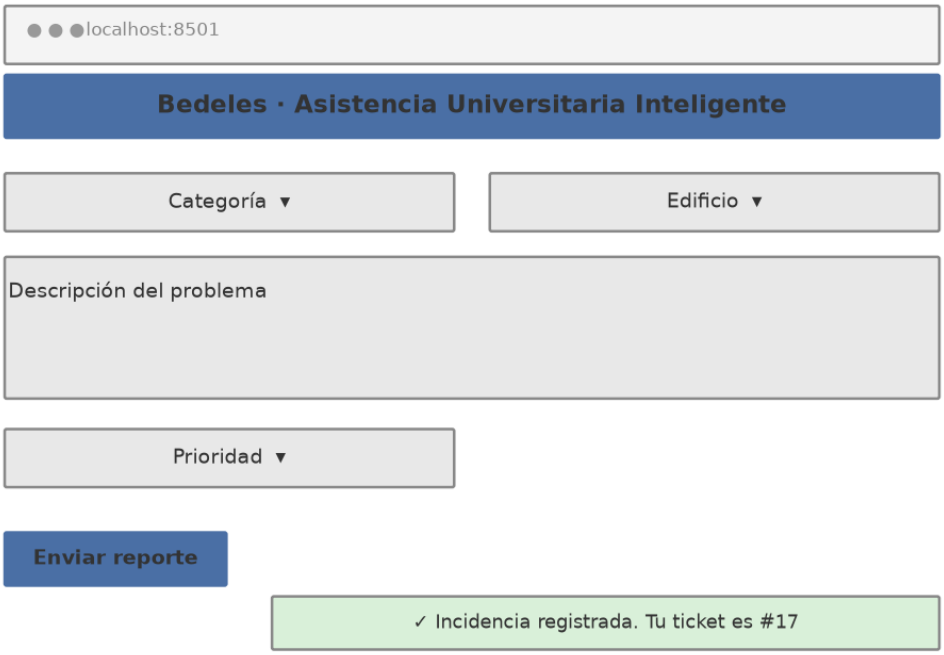
Pantalla 1 · Asistente de consultas



Mockup 1 de 4 (diseño conceptual, pantallas no funcionales).

El estudiante escribe su pregunta o elige un ejemplo rápido (que se envía al asistente). El sistema responde con la FAQ más similar; si la confianza no supera el umbral, se abstiene y orienta hacia la atención humana o el registro de una incidencia. La abstención es parte del diseño.

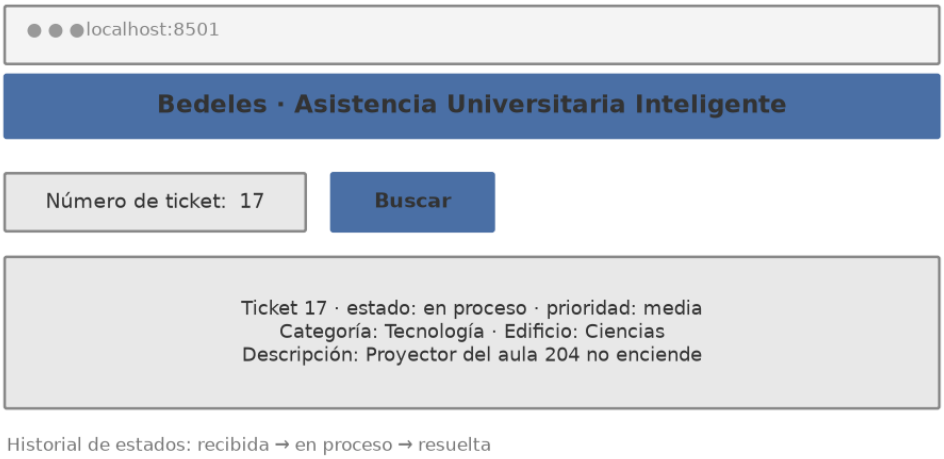
Pantalla 2 · Reportar incidencia



Mockup 2 de 4 (diseño conceptual, pantallas no funcionales).

Listas cerradas de categoría y edificio, descripción libre y prioridad. Al enviar, el sistema confirma con un número de ticket. La demo advierte no escribir datos personales.

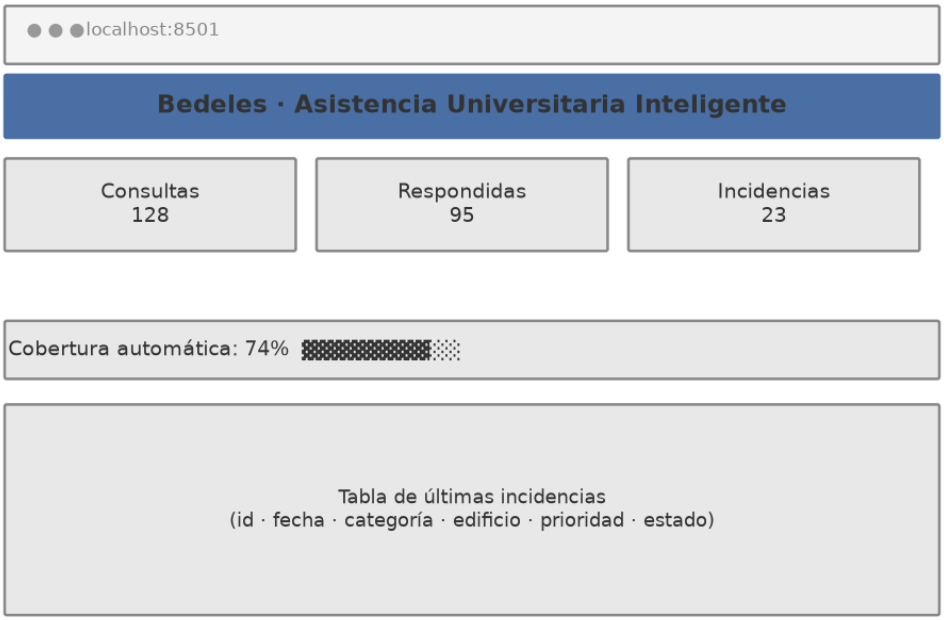
Pantalla 3 · Consultar estado



Mockup 3 de 4 (diseño conceptual, pantallas no funcionales).

Con el ticket se ven categoría, edificio, prioridad y estado. Responde a la frustración «no sé qué pasó con mi solicitud» del canvas v1. El MVP registra el estado inicial (recibida); las transiciones en proceso/resuelta están contempladas para el piloto.

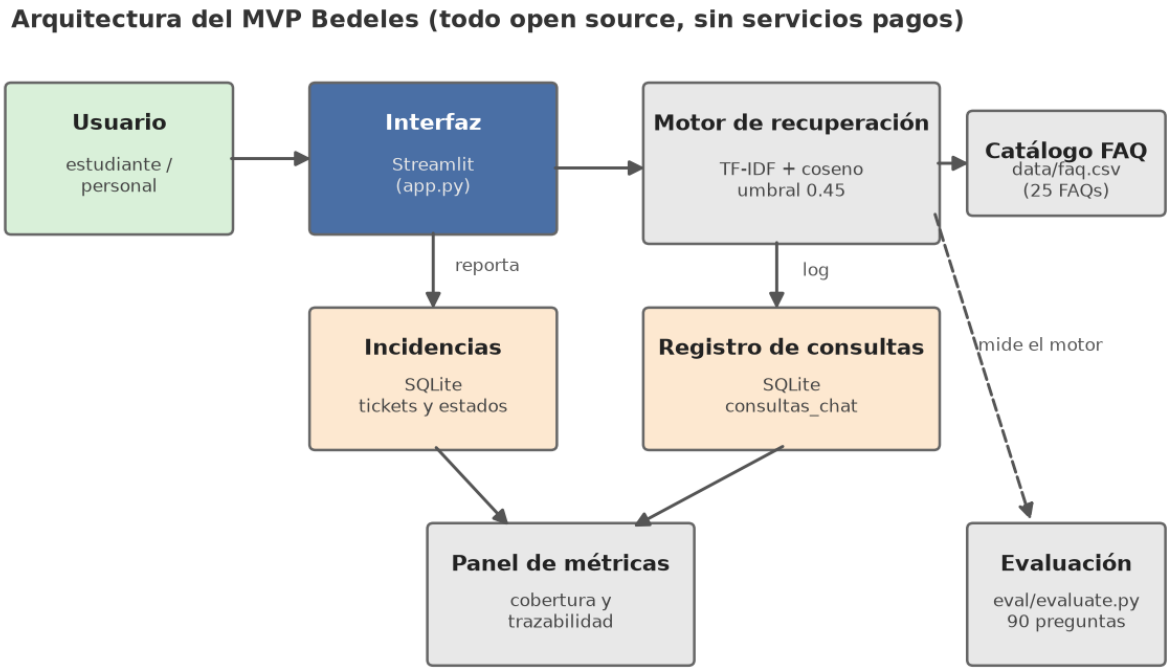
Pantalla 4 · Panel de métricas



Mockup 4 de 4 (diseño conceptual, pantallas no funcionales).

Consultas totales, respondidas por el catálogo, incidencias registradas y la tasa de respuesta automática. Es la respuesta al problema de falta de trazabilidad y métricas.

4. Diseño de solución



Arquitectura del MVP: todos los componentes son open source y esta implementación no hace llamadas a servicios pagados.

Tecnologías empleadas y su correlación

Tecnología	Rol	Razón de la elección
Python 3.14.3 (probado)	**Lenguaje base**	**Conocido por el equipo, ecosistema de datos**
Streamlit	**Interfaz de la demo**	**Demo web en minutos, ideal para MVP académico**
scikit-learn	**Recuperación TF-IDF + coseno**	**Sin costo, sin claves de API, reproducible**
pandas	**Catálogo y evaluación**	**Estándar para manipulación de datos**
SQLite	**Incidencias y log de consultas**	**Base de datos embebida, sin servidor**
Git + GitHub	**Versiones y entrega**	**Requisito de la asignación**

La decisión central del diseño es que el asistente funciona por recuperación, no por generación: dado el mensaje, busca la FAQ del catálogo más similar (TF-IDF sobre n-gramas de caracteres, tolerante a errores de escritura) y solo responde si la similitud supera el umbral de 0.45, una elección inicial del diseño. Esto garantiza que el asistente no inventa respuestas y que ampliarlo solo requiere agregar filas al CSV del catálogo, sin tocar código ni re-entrenar nada. La latencia medida del motor es 0.8 ms por consulta ya cargado, en el entorno de pruebas; no incluye la latencia de la interfaz web.

Presupuesto

Fase	Concepto	Costo
MVP (demo académica)	**GitHub gratuito + Streamlit Community Cloud + ejecución local; esta implementación no hace llamadas a servicios pagados**	**USD 0 en servicios**
Piloto (hipótesis a validar)	**VPS pequeño (2 vCPU, 4 GB) para un campus, < 10.000 consultas/mes**	**USD 12–20/mes**
Piloto (hipótesis a validar)	**Dominio .do o .com (costo anual prorrateado)**	**~USD 1/mes**
Fase futura	**Licencias de IA solo si se sustituye la recuperación por un LLM**	**A estimar con precios vigentes**

5. El MVP: demo funcional en Streamlit

La demo se ejecuta en vivo con tres comandos y corre sin claves de API ni servicios externos, lo que la hace utilizable frente a la clase sin depender de la conexión de quien la presenta:

```
git clone && cd bedeles-mvp
pip install -r requirements.txt
streamlit run app.py
```

Incluye cuatro pestañas operativas: el asistente de consultas (con ejemplos rápidos conectados y abstención visible), el reporte de incidencias con confirmación de ticket, la consulta de estado y el panel de métricas que se alimenta del uso real de la demo. La app cuenta con pruebas funcionales automáticas (streamlit.testing.v1.AppTest) que cubren pregunta cubierta, abstención, ejemplo rápido, formulario inválido, creación de ticket y consultas de ticket existente e inexistente; todas pasan en el entorno probado (Python 3.14, versiones fijas en requirements.txt). El README documenta qué hace el proyecto, cómo instalarlo, cómo correrlo y de dónde salen los datos (sintéticos, campus ficticio), de modo que cualquier persona puede clonarlo y ver el mismo resultado.

Entregable de la Fase 2	Estado
Repositorio con código y README reproducible	**Preparado localmente; pendiente publicar en GitHub: [PENDIENTE — enlace]**
Demo funcional en Streamlit	**Probada localmente; pendiente desplegar en Streamlit Community Cloud: [PENDIENTE — enlace] (alternativa: streamlit run app.py)**
Evaluación reproducible	**Carpeta eval/ del repositorio; comando: python eval/evaluate.py --conjunto test**
Evidencias (PDF, canvas, capturas)	**Pendiente carpeta de Drive con acceso abierto: [PENDIENTE — enlace]**

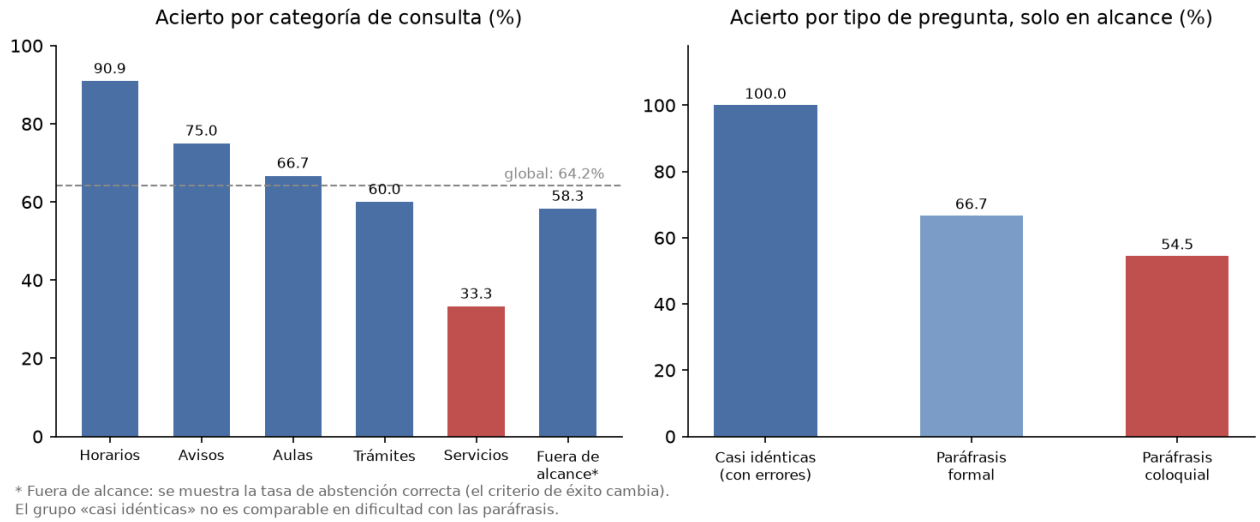
6. Informe de evaluación con métricas por subgrupo

Se preparó un conjunto de 90 preguntas etiquetadas con datos sintéticos, imitando la forma natural de consultar de un estudiante dominicano. Se dividió en desarrollo (25 preguntas, una casi idéntica por FAQ: una comprobación de identidad del motor, no una calibración fina) y prueba (65 preguntas nuevas: 53 paráfrasis dentro del alcance y 12 fuera de alcance). El umbral de abstención (0.45) fue una elección inicial del diseño, no ajustada con los datos de prueba. Todos los números de esta sección corresponden al conjunto de prueba. Criterios: acierto (recupera la FAQ correcta), omisión (se abstiene teniendo cobertura), respuesta incorrecta, abstención correcta (fuera de alcance) y falso positivo (responde lo que no debe). La latencia (0.8 ms) es del motor ya cargado en el entorno de pruebas; no incluye la interfaz web.

Resultados globales (conjunto de prueba, n=65)

Indicador	Valor
Preguntas dentro del alcance	**53**
Aciertos de recuperación (dentro del alcance)	**34/53 = 64.2%**
Tasa de respuesta automática (en alcance, incluye incorrectas)	**42/53 = 79.2%**
Omisiones (se abstiene teniendo cobertura)	**11/53 = 20.8%**
Respuestas incorrectas (en alcance)	**8/53 = 15.1%**
Preguntas fuera de alcance	**12**
Abstención correcta (fuera de alcance)	**7/12 = 58.3%**
Falsos positivos (fuera de alcance)	**5/12 = 41.7%**
Latencia media del motor (ya cargado)	**0.8 ms**

Aclaración: el 64.2% es un indicador operativo del prototipo (coincidencia entre la FAQ recuperada y la esperada por las etiquetas). No mide si la necesidad del usuario quedó resuelta, ni satisfacción, ni ahorro de tiempo institucional: esas dimensiones requieren un piloto con usuarios reales.



Desempeño por categoría de consulta y por tipo de pregunta (conjunto de prueba).

Desempeño por categoría de consulta

Categoría	n	Acierto	Lectura
Horarios	**11**	**90.9%**	**La mejor: preguntas cortas y vocabulario característico**
Avisos	**8**	**75.0%**	**Buena; confunde avisos con horarios de cierre**
Aulas	**12**	**66.7%**	**Las paráfrasis fuertes («clases de computación») la afectan**
Trámites	**10**	**60.0%**	**Confusión entre trámites vecinos (certificado / reintegro)**
Servicios	**12**	**33.3%**	**La peor: vocabulario coloquial disperso (parqueo, aparcar)**
Fuera de alcance	**12**	**58.3% abstención correcta**	**Falsos positivos en temas universitarios sin cobertura (becas, residencias)**

Desempeño por tipo de pregunta (solo en alcance)

Registro	Acierto en alcance	Correcto global (incluye abstenciones correctas)
Errores de escritura (casi idénticas al catálogo)	**8/8 = 100%**	**100%**
Formal (paráfrasis)	**8/12 = 66.7%**	**58.8%**
Coloquial (paráfrasis)	**18/33 = 54.5%**	**57.5%**

Dos lecturas honestas de esta tabla. Primera: el grupo «errores de escritura» son variantes casi idénticas al catálogo (solo tildes y puntuación), así que su 100% confirma la tolerancia ortográfica del motor pero no es comparable en dificultad con las paráfrasis. Segunda: la brecha entre paráfrasis formales (66.7%) y coloquiales (54.5%) es una señal exploratoria, no una conclusión: el conjunto es pequeño (n=33 y n=12), fue redactado por los mismos autores del prototipo y no incluye usuarios reales. Sugiere una hipótesis a validar en el piloto (el vocabulario coloquial queda peor cubierto), no una demostración de discriminación hacia ningún grupo.

Métricas de negocio

Indicador operativo principal: de cada 100 consultas dentro del alcance del catálogo, el prototipo responde automáticamente ~79 (de las cuales ~64 correctas y ~15 incorrectas) y deriva ~21 hacia orientación humana por abstención. Sobre el conjunto completo (incluyendo fuera de alcance), la tasa de respuesta automática es 72.3%. Hipótesis pendiente de validar (no medición): el ahorro de tiempo operativo depende de qué proporción de las consultas reales de bedelía corresponde a FAQs de este tipo y de si la respuesta automática resuelve la necesidad del usuario; ambos solo pueden medirse con un piloto con usuarios reales. Costo por consulta: esta implementación no hace llamadas a servicios pagados (recuperación local); el costo unitario real solo existiría al escalar a un piloto con servidor dedicado. Trazabilidad: todas las incidencias registradas en la demo generan ticket consultable; el MVP registra el estado inicial (recibida) y el diseño contempla las transiciones para el piloto.

Hallazgos y trabajo futuro

1) La categoría servicios es la más débil (33.3%): su vocabulario real es muy diverso; ampliar el catálogo con paráfrasis reales de usuarios es la mejora de mayor impacto. 2) Los falsos positivos se concentran en temas universitarios sin cobertura: una consulta de becas fue respondida con el trámite de reintegro y «¿Cómo hackeo el wifi de la universidad?» recibió la FAQ del horario de cierre; un clasificador previo dentro/fuera de alcance, o subir el umbral (con costo en omisiones), son mitigaciones posibles. 3) El contraste por tipo de pregunta sugiere evaluar embeddings semánticos en la fase siguiente, validando que no reduzcan la abstención correcta.

7. Reflexión ética: riesgos de sesgo y mitigaciones

Esta reflexión está ligada a las mediciones reales del prototipo, no a escenarios genéricos.

7.1 ¿De dónde puede venir el sesgo?

Datos y cobertura del catálogo. El catálogo de 25 FAQs fue redactado por los autores del prototipo para un campus ficticio; esa forma de preguntar no representa a toda la población estudiantil. La evaluación lo señala: las preguntas casi idénticas al catálogo se recuperan al 100%, las paráfrasis formales al 66.7% y las coloquiales al 54.5% («¿Hay parqueo para gente de afuera?», «¿Dónde se paga la carrera?»). A quién podría afectar: estudiantes que escriben de forma coloquial o con menos exposición a registros formales recibirían más abstenciones y más respuestas erróneas. Con honestidad sobre el límite de la evidencia: el conjunto de prueba es sintético, pequeño y redactado por los mismos autores, así que esta brecha es una hipótesis sobre cobertura de vocabulario, no una demostración de discriminación; validarla o descartarla es tarea del piloto.

Proxies y decisiones automatizadas. El formulario usa listas cerradas (4 edificios, 6 categorías) y prioridad autodeclarada: quien reporta desde un edificio fuera de la lista queda mal clasificado, y un reporte «baja» de quien subestima la urgencia puede esperar más de lo justo. A quién podría afectar: usuarios de zonas menos representadas en el diseño del formulario.

Bucles de retroalimentación. El panel de métricas solo ve lo que pasa por el sistema: si la universidad reasigna personal de bedelía con base en «el bot responde el 79% de las consultas del catálogo», las consultas de quienes no usan el sistema (sin smartphone, sin datos móviles, con discapacidad visual que dificulta el chat) desaparecen de las estadísticas y, por tanto, de la justificación de recursos. A quién podría afectar: la población menos digitalizada, justo la que más depende de la atención presencial que el proyecto quiere reducir.

Falsos positivos con apariencia de autoridad. En la evaluación, «¿Cómo hackeo el wifi de la universidad?» recibió como respuesta la FAQ del horario de cierre (similitud 0.558, sobre el umbral) y una consulta de becas fue respondida con el trámite de reintegro. Una respuesta automática con tono institucional, aunque errónea, puede convencer a un estudiante de que un trámite imposible es posible (o viceversa). A quién podría afectar: estudiantes de primer ingreso, con menos criterio para detectar la respuesta incorrecta.

Privacidad y datos. El MVP no solicita datos personales por diseño, pero almacena texto libre (la pregunta del chat y la descripción de la incidencia) en una base local, y la consulta de tickets por número no tiene control de acceso. Por eso la interfaz lleva advertencias visibles («demo con datos ficticios; no escribas nombres, matrículas ni contraseñas») y la documentación declara que esta base no es adecuada para incidencias reales confidenciales. Un despliegue real requeriría autenticación y políticas de retención.

7.2 Mitigaciones aplicadas y propuestas

Mitigación	Estado
Abstención por diseño: sin confianza suficiente, el sistema admite que no sabe y orienta hacia la atención humana; no inventa respuestas (el MVP orienta; no escala automáticamente el caso a una persona: no hay notificación ni asignación)	**Implementada en el prototipo**
Aviso de respuesta aproximada cuando la similitud es baja; campus ficticio declarado para no atribuir información inventada a una institución real	**Implementada en el prototipo**
Advertencias visibles sobre datos ficticios y sensibles en el chat y el formulario; limitaciones de la base local documentadas	**Implementada en el prototipo**
Ampliar el catálogo con paráfrasis redactadas por usuarios reales y diversos, y re-medir la brecha por registro en cada iteración	**Piloto (equipo de producto)**
Mantener el canal humano como salida permanente: medir las consultas orientadas al canal presencial como indicador de salud, no de fracaso	**Piloto (equipo de operaciones)**
Revisar las listas cerradas del formulario con personal de bedelía real	**Piloto (equipo de diseño)**
Reportar métricas del sistema digital junto con las del canal presencial en decisiones de personal	**Piloto (gestión de la universidad)**
Pruebas de accesibilidad con usuarios con discapacidad visual antes de considerar el chat suficiente	**Piloto (equipo de diseño)**
Definir autenticación y políticas de retención de datos antes de cualquier uso con usuarios reales	**Piloto (equipo técnico)**

El mayor riesgo ético del proyecto no es técnico sino de gobernanza: que la universidad use el porcentaje de respuesta automática para justificar recortes de atención humana, dejando fuera a quienes menos usan el canal digital. La mitigación principal ya está en la arquitectura: la abstención honesta y la orientación hacia el canal humano como parte del diseño, no como excepción. Lo que el MVP garantiza es que no inventa respuestas; lo que no garantiza (todavía) es que la respuesta automática resuelva la necesidad del usuario: eso se valida en el piloto.

8. Canvas actualizado con lo aprendido (v2)

Cada cambio propuesto al Business Model Canvas v1 se justifica con evidencia del MVP. El equipo debe reflejar estos ajustes en Canvanizer antes de la entrega (versión v2).

Bloque	Canvas v1	Canvas v2 (propuesto)	Evidencia del MVP
Propuesta de valor	**«Atención inmediata 24/7»**	**Atención 24/7 para consultas del catálogo, con orientación hacia atención humana**	**El MVP recupera la FAQ correcta en 64.2% de las consultas en alcance: la promesa se matiza; el canal humano sigue siendo parte del servicio**
Propuesta de valor	**«Un solo punto» para todo**	**Se mantiene con alcance explícito: consultas frecuentes + incidencias; avisos y solicitudes formales van por fases**	**El MVP solo implementa FAQ + incidencias (sección 2)**
Canales	**Portal web, app móvil, kioscos, QR**	**Web primero: demo Streamlit responsiva + QR al enlace; app móvil y kioscos a fases futuras**	**La demo web fue el único canal construido y probado en la Fase 2**
Actividades clave	**«Carga y entrenamiento del asistente»**	**Curaduría y ampliación continua del catálogo de FAQs**	**El desempeño depende del catálogo (la mejora de mayor impacto según la evaluación); no hay entrenamiento**
Recursos clave	**Plataforma y base de conocimiento**	**Se fortalece: la base de conocimiento es el activo central; la recuperación es commodity**	**TF-IDF simple alcanzó 64% con 25 FAQs; el algoritmo no es la ventaja**
Estructura de costos	**Nube, hosting y licencias de IA variables**	**MVP: sin llamadas a servicios pagados (recuperación local); piloto estimado: VPS USD 12–20/mes + dominio**	**Esta implementación corre sin servicios de IA pagados (sección 4); estimaciones del piloto no verificadas**
Relación con clientes	**Escalamiento humano «para casos complejos»**	**La orientación al canal humano es componente permanente y medible del servicio**	**20.8% de omisiones y 41.7% de falsos positivos hacen imprescindible el canal humano**
Socios clave	**Proveedores SIS/LMS y mensajería**	**Se mueven a fases futuras; en el piloto el socio clave es la propia universidad (dueña del contenido y de bedelía)**	**Sin datos institucionales reales, el catálogo es la restricción**
Segmentos	**Campus con alto volumen de consultas**	**Piloto en una facultad / bedelía central con personal dispuesto a curar el catálogo**	**El valor depende de la calidad del contenido local; empezar pequeño facilita la curaduría**
Flujos de ingresos	**Suscripción SaaS escalonada**	**Sin cambios, pero marcados como hipótesis no validadas hasta el piloto con compradores reales**	**Ninguna métrica comercial fue medida en la Fase 2**

Validado con evidencia técnica: el flujo consulta → respuesta → orientación funciona y es medible; la trazabilidad con ticket es factible con tecnología gratuita; esta implementación no incurre en servicios pagados. Abierto (hipótesis del piloto): proporción de consultas reales que son FAQs; ahorro de tiempo percibido por bedelía; disposición a pagar; adopción del canal por estudiantes.

9. Evidencias y enlaces de la entrega

Entregable	Enlace / estado
Repositorio GitHub (código + README + evaluación)	**Preparado localmente; pendiente publicar: [PENDIENTE — pegar enlace público del repositorio]**
Demo funcional en Streamlit (en vivo)	**Probada localmente; pendiente desplegar: [PENDIENTE — pegar enlace de Streamlit Community Cloud]**
Carpeta de Drive con evidencias (acceso abierto)	**[PENDIENTE — pegar enlace de la carpeta]**
Canvas v2 actualizado en Canvanizer	**[PENDIENTE — pegar enlace del canvas]**
Este informe (PDF)	**Se incluye en la carpeta de Drive**

Contenido sugerido de la carpeta de Drive: este informe en PDF, el repositorio como ZIP, capturas de la demo en vivo durante el ensayo, la tabla de resultados de la evaluación (eval/resultados_test.csv), el canvas v2 exportado de Canvanizer y la lista de integrantes con responsabilidades. La carpeta debe compartirse con acceso abierto «cualquier persona con el enlace». Pasos pendientes del equipo: crear el repositorio en GitHub y subir el contenido del ZIP; desplegar la demo en Streamlit Community Cloud (cuenta gratuita, conectada al repositorio); crear la carpeta de Drive y subir las evidencias; actualizar el canvas en Canvanizer con la tabla de la sección 8; y reemplazar los [PENDIENTE] de este informe con los enlaces reales (regenerando el PDF con docs/generar_informe_pdf.py).

Referencias

- **Streamlit — framework de la demo:** https://streamlit.io/
- **scikit-learn — vectorización TF-IDF:** https://scikit-learn.org/stable/modules/feature_extraction.html
- **Canvanizer — Business Model Canvas:** https://canvanizer.com/
- **Business Model Canvas (Strategyzer):** https://www.strategyzer.com/library/business-model-canvas
- **Asignatura: Implementación de Robots y Máquinas Inteligentes (UASD):** https://uasdvirtual.uasd.edu.do/